



## Why I Chose Computer Vision

I chose Computer Vision because it is one of the most exciting and impactful fields of Artificial Intelligence. It enables computers to understand and interpret visual information from images and videos, allowing them to perform tasks that traditionally required human vision. I have always been interested in building intelligent systems that can solve real-world problems, and Computer Vision provides the opportunity to work on applications such as healthcare, security, robotics, and autonomous systems.

During my academic journey, I worked on projects related to image processing and machine learning, which further increased my interest in this field. Joining the Computer Vision Fellowship allows me to strengthen my practical skills and gain hands-on experience with modern AI technologies used in industry. An image is ultimately a structured matrix of data, and bridging that technical map into human-like intelligence is incredibly profound.

*"Giving machines the ability to see is not just a technical problem — it is the foundation of every system that must actively interpret and interact with the physical world."*

## Applications That Interest Me Most

### 01 / MEDICAL

#### Image Analysis

AI models assisting doctors by detecting conditions like pneumonia or tumors from X-rays and MRI scans to automate diagnostic support pipelines.

### 02 / SECURITY

#### Object Detection

Intelligent surveillance systems capable of framing people, vehicles, safety gear, or suspicious anomalies to maximize public safety standards.

### 03 / AUTONOMOUS

#### Smart Vehicles

Real-time image processing systems for lane detection, obstacle mapping, sign recognition, and pedestrian safety in self-driving frameworks.

## What I Learned This Week

### PIPELINE CORE

**Modern Computer Vision Architecture:** Mastered the comprehensive multi-stage pipeline driving production environments from acquisition, structural preprocessing, all the way to predictive model inferences.

#### OPENCV BASICS

**Matrix Manipulations:** Acquired practical clarity on spatial algorithms covering grayscale operations, blur/smoothing windows, Canny edge gradients, and pixel threshold levels.

#### DEEP LEARNING

**CNNs & YOLO Framework:** Studied Convolutional Neural Networks and object detection paradigms. Learned how deep feature extractions empower single-shot architectural detectors to infer bounding boxes.

#### ENGINEERING

**Dataset Management & Tooling:** Learned best practices for structuring annotation files, handling data version control, and maintaining code states using professional Git and GitHub development workflows.

## Biggest Challenge & How I Solved It

### ⚠ THE PROBLEM — SHIFTING THEORY TO PRODUCTION

The core challenge was translating theoretical knowledge into engineering realities. Connecting backend processing blocks (Flask + OpenCV) to coordinate dynamically with interactive frontend views required synchronous state management, structured directory organizations, and absolute precision over image payload matrices.

### ✓ THE SOLUTION — MILESTONE DECOUPLING

I systematically broke down the system architecture into separate atomic components. Instead of coding everything simultaneously, I decoupled OpenCV algorithms into simple command-line scripts first, wrapped them in predictable web-endpoint routes, and applied Git to systematically track stability milestones.

```
# Stateless processing sample pattern developed this week
def process_image_stream(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    return cv2.Canny(blurred, 50, 150)
```

This methodical decoupling removed the complexity barrier, enabling me to firmly grasp how multi-stage pixels interface with end-user dashboards.